

자연어처리 개념 정리

한국어 챗봇 프로젝트로 배우는 NLP 기본 용어

항목	내용
이 문서는	자연어처리를 처음 접하는 사람이 용어를 이해할 수 있도록 정리한 학습 자료
특징	모든 용어에 "이 프로젝트에서 어떻게 썼는지"를 실제 수치와 함께 붙였다
기준 프로젝트	한국어 챗봇 응답 생성(Q→A) — 노트북 01~08
읽는 법	처음부터 순서대로 읽으면 프로젝트 진행 순서와 같다. 용어만 찾고 싶으면 마지막 색인 사용
작성일	2026-09-20

0. 이 문서를 읽는 법

자연어처리(NLP) 용어는 서로 얽혀 있어서, 사전처럼 가나다순으로 읽으면 오히려 이해가 안 됩니다. 예를 들어 "임베딩"을 이해하려면 먼저 "토큰"을 알아야 하고, "BLEU"를 이해하려면 "n-gram"을 알아야 합니다.

그래서 이 문서는 실제 프로젝트를 진행하는 순서대로 용어를 배치했습니다. 데이터를 준비하고 → 문장을 쪼개고 → 숫자로 바꾸고 → 모델을 만들고 → 학습시키고 → 문장을 생성하고 → 채점하고 → 틀린 걸 분석하는 순서입니다. 이 순서를 따라가면 앞에서 배운 용어가 뒤에서 자연스럽게 재사용됩니다.

용어 항목의 구성

각 용어는 다음 세 부분으로 되어 있습니다.

- 한 줄 정의 — 이것만 기억해도 되는 핵심 문장
- 설명 — 왜 필요한지, 어떻게 작동하는지를 비유와 함께 풀어쓴 부분
- 이 프로젝트에서 (파란 상자) — 실제로 어떤 값을 썼고 어떤 결과가 나왔는지

파란 상자만 따로 읽어도 됩니다. 프로젝트 보고서를 이미 읽었다면, 파란 상자가 "그 숫자가 사실은 이런 뜻이었구나"를 연결해 주는 역할을 합니다.

이 프로젝트가 풀 문제 한 줄 요약

질문(Q)을 입력하면 어울리는 답변(A)을 만들어내는 한국어 챗봇을 만드는 것입니다. 예를 들어 "오늘 생각보다 춥네"라고 입력하면 "감기 조심하세요."처럼 답하는 모델입니다.

이때 두 가지 방식을 비교했습니다. 하나는 맨땅에서 직접 만든 모델(베이스라인), 다른 하나는 남이 미리 한국어를 잔뜩 학습시켜둔 모델을 가져와 우리 데이터로 다듬은 것(전이학습)입니다. 이 대비가 프로젝트 전체를 관통하는 줄거리입니다.

1. 큰 그림 — 자연어처리란 무엇인가

자연어처리 (NLP, Natural Language Processing)

사람이 쓰는 말과 글을 컴퓨터가 다루게 만드는 기술 분야.

"자연어"는 한국어·영어처럼 사람이 자연스럽게 쓰는 언어를 말합니다. 프로그래밍 언어 같은 "인공어"와 구별하려고 붙인 이름입니다. 컴퓨터는 숫자만 다룰 수 있으므로, NLP의 상당 부분은 "글을 어떻게 숫자로 바꿀 것인가"와 "그 숫자로 무엇을 할 것인가"에 대한 이야기입니다.

이 프로젝트에서 — 한국어 대화 문장 11,823쌍을 숫자로 바꿔 모델에 넣고, 다시 한국어 문장으로 되돌리는 전 과정을 다뤘습니다.

분류 태스크 / 생성 태스크 (Classification / Generation)

정해진 보기 중 하나를 고르는 문제 vs 문장을 새로 만들어내는 문제.

분류는 "이 리뷰는 긍정인가 부정인가?"처럼 답이 정해진 보기 안에 있는 문제입니다. 보기가 2개면 둘 중 하나만 맞으면 되니 채점도 간단합니다.

생성은 "이 질문에 답해봐"처럼 답을 새로 써내야 하는 문제입니다. 훨씬 어렵고, 무엇보다 정답이 하나가 아닙니다. "배고파"에 대해 "뭐 좀 드세요"도 "얼른 식사하세요"도 다 좋은 답입니다. 이 성질이 나중에 평가(채점)를 까다롭게 만드는 근본 원인이 됩니다.

이 프로젝트에서 — 이 프로젝트는 생성 태스크입니다. 데이터에 감정 label(일상/이별/사랑)이 있어서 분류 문제로 풀 수도 있었지만, 과제 요구사항이 "응답 생성"이라 label은 학습에 쓰지 않고 평가를 나눠 보는 용도로만 썼습니다.

조건부 생성 (Conditional Generation)

아무 문장이나 만드는 게 아니라, 주어진 입력에 "맞춰서" 문장을 만드는 것.

그냥 생성이 "소설을 한 편 써봐"라면, 조건부 생성은 "이 질문에 답해봐", "이 글을 요약해봐", "이 영어 문장을 한국어로 옮겨봐"처럼 입력이 출력을 제약하는 경우입니다. 기계번역·요약·챗봇이 전부 여기 속하고, 그래서 셋은 기술적으로 아주 비슷한 구조를 씁니다.

이 프로젝트에서 — Q(질문)를 조건으로 A(답변)를 생성합니다. 이 문제 정의가 "그러면 기계번역과 같은 구조를 쓰면 되겠다"는 모델 선택으로 바로 이어졌습니다(5장).

말뭉치 (Corpus, 코퍼스)

분석·학습에 쓰려고 모아둔 텍스트 데이터 덩어리.

그냥 "텍스트 데이터 모음"입니다. 위키백과 전체, 뉴스 기사 10만 건, 영화 리뷰 5만 개 같은 것들이 다 말뭉치입니다. 말뭉치의 크기와 성격이 모델 성능을 크게 좌우합니다 — 뉴스로만 학습한 모델은 구어체 채팅을 잘 다루지 못합니다.

이 프로젝트에서 — `ChatbotData.xlsx` — 한국어 일상 대화 Q/A 쌍 11,823건. 짧은 구어체 문장이라는 점이 토큰라이저 선택(3장)과 모델 크기 결정(5장)에 계속 영향을 줬습니다.

2. 데이터 다루기 — 학습 전에 정리하는 일

모델을 만들기 전에 데이터를 살펴보고 정리하는 단계입니다. 지루해 보이지만, 여기서 측정한 숫자들이 뒤 단계의 설계 근거가 되기 때문에 건너뛸 수 없습니다.

탐색적 데이터 분석 (EDA, Exploratory Data Analysis)

본격적인 작업 전에 데이터가 어떻게 생겼는지 훑어보는 단계.

데이터가 몇 건인지, 빠진 값은 없는지, 중복은 없는지, 문장 길이는 어느 정도인지를 확인합니다. 요리로 치면 재료 손질 전에 재료를 펼쳐놓고 상태를 확인하는 단계입니다. 여기서 얻은 숫자가 나중에 "왜 그 값을 골랐나요?"라는 질문의 답이 됩니다.

이 프로젝트에서 — 01번 노트북. 결측치 0건, (Q,A) 완전 중복 73건, 문장 길이 분포 등을 측정했고, 여기서 나온 길이 95 percentile이 나중에 모델의 입력 길이 상한을 정하는 근거가 되었습니다.

백분위수 (Percentile)

데이터를 작은 순서로 줄 세웠을 때 특정 위치에 오는 값.

"95 percentile이 25"라는 말은 전체의 95%가 25 이하라는 뜻입니다. 평균과는 다릅니다. 평균은 극단값 하나에 크게 휘돌리지만(연봉 평균에 역대 연봉자 한 명이 끼면 확 올라감), percentile은 순서만 보기 때문에 안정적입니다.

이 프로젝트에서 — 질문 길이 95p = 25음절, 답변 95p = 27음절. 최댓값(56, 76)이 아니라 95p를 기준으로 삼은 이유는, 최댓값에 맞추면 소수의 긴 문장 때문에 나머지 대부분의 짧은 문장에 쓸데없는 빈칸(padding)이 잔뜩 붙어 계산이 낭비되기 때문입니다.

학습/검증/테스트 분할 (Train / Validation / Test Split)

데이터를 세 덩어리로 나눠서, 학습용·중간점검용·최종채점용으로 따로 쓰는 것.

시험 공부에 비유하면 이렇습니다.

- train(학습) = 공부하는 문제집. 모델이 이걸 보고 배웁니다.
- validation(검증) = 모의고사. 학습 중간에 "지금 잘 되고 있나?" 확인하고, 언제 멈출지 결정하는 데 씁니다.
- test(테스트) = 실제 수능. 딱 한 번, 맨 마지막에만 봅니다.

왜 나눌까요? 공부한 문제집을 그대로 시험 내면 누구나 100점이기 때문입니다. 모델이 외운 것인지 이해한 것인지 구별하려면 한 번도 본 적 없는 문제로 채점해야 합니다.

이 프로젝트에서 — 11,750건을 8:1:1로 나눠 train 9,400 / val 1,175 / test 1,175건을 만들었습니다. 학습 중 val loss를 보며 멈출 시점을 정했고, test는 07번 최종 평가에서만 썼습니다.

층화 추출 (Stratified Sampling, stratify)

나눌 때 각 종류의 비율이 원본과 같도록 맞춰서 나누는 것.

데이터에 종류가 섞여 있는데 그냥 무작위로 자르면, 운 나쁘면 한쪽에 특정 종류가 몰릴 수 있습니다. 층화 추출은 종류별로 따로 나눠서 합치기 때문에 어느 덩어리든 원본과 같은 구성비가 됩니다.

이 프로젝트에서 — label(감정) 비율을 유지하며 나눴습니다. 원본이 일상 5,290 / 이별 3,570 / 사랑 2,963으로 치우쳐 있어서, 그냥 잘랐다면 test에 특정 감정이 몰릴 수 있었습니다. 실제로 test set은 일상 529 / 이별 350 / 사랑 296으로 원본 비율이 그대로 유지됐고, 덕분에 08번에서 감정별 성능 비교가 공정하게 가능했습니다.

과적합 (Overfitting)

모델이 학습 데이터를 "외워버려서" 처음 보는 데이터에는 오히려 못하게 되는 현상.

기출문제만 통째로 외운 학생과 같습니다. 문제집은 만점인데 새 문제는 못 풉니다. 증상은 명확합니다 — train loss는 계속 내려가는데 val loss는 오히려 올라가기 시작합니다. 그 갈라지는 지점이 "외우기 시작한 시점"입니다.

이 프로젝트에서 — KoBART 파인튜닝에서 뚜렷하게 관찰됐습니다. train loss가 0.44까지 내려가는 동안 val loss는 5 epoch째부터 반등했습니다. 그래서 조기 종료(6장)로 val loss가 가장 낮았던 시점의 모델을 저장해 썼습니다.

3. 토큰화 — 문장을 어떻게 쪼갤 것인가

컴퓨터는 "안녕하세요"라는 문장을 통째로 다루지 못합니다. 의미 있는 조각으로 쪼갠 뒤 각 조각에 번호를 붙여야 합니다. 이 "쪼개기"가 토큰화이고, 어떻게 쪼개느냐에 따라 모델 성능이 크게 달라집니다.

토큰 / 토큰화 (Token / Tokenization)

문장을 처리 단위로 쪼갠 조각이 토큰, 쪼개는 작업이 토큰화.

영어는 띄어쓰기로 자르면 대충 단어가 되지만("I love you" → I / love / you), 한국어는 그렇게 간단하지 않습니다. "학교에서", "학교를", "학교가"가 전부 다른 조각이 되어버리는데, 사람이 보기엔 다 같은 "학교"입니다. 이 문제를 어떻게 다루느냐가 한국어 NLP의 핵심 고민입니다.

이 프로젝트에서 — 어절·음절·형태소·자모 네 가지를 실제로 측정해 비교한 뒤(아래 표), 최종적으로 서브워드(BPE)를 모델 입력용으로 채택했습니다.

한국어를 쪼개는 네 가지 방법을 실제로 비교한 결과입니다. 같은 문장이 방식에 따라 조각 수가 7배까지 차이 납니다.

방식	평균 토큰 수	예: "오늘 저녁에 뭐 먹을지"	특징
어절 (띄어쓰기)	3.53	오늘 / 저녁에 / 뭐 / 먹을지	가장 단순하지만 조사가 붙어 있어 종류가 폭증
형태소 (Okt)	5.26	오늘 / 저녁 / 에 / 뭐 / 먹을지	의미 단위 보존. 신조어·오타에 약함
음절 (글자)	10.31	오 / 늘 / 저 / 녁 / 에 / ...	종류는 적지만 글자 하나엔 의미가 거의 없음
자모 (ㄱ ㅏ)	24.54	ㅇ / ㅏ / ㄴ / ㅡ / ㄹ / ...	오타에 가장 강하지만 너무 길어져 계산량 폭증

표 3-1. 02번 노트북 실측. 토큰이 많을수록 모델의 계산량이 늘어나므로, 무조건 잘게 쪼개는 게 좋은 것은 아닙니다.

형태소 / 형태소 분석기 (Morpheme / Morphological Analyzer)

뜻을 가진 최소 단위가 형태소, 그것으로 쪼개주는 도구가 형태소 분석기.

"먹었어요"를 먹 + 었 + 어요로 나누는 식입니다. "먹"이라는 의미 덩어리와 과거를 뜻하는 "었", 높임을 뜻하는 "어요"를 분리하는 것이죠. 이렇게 하면 "먹었어요/먹습니다/먹는다"가 모두 "먹"을 공유하게 되어 모델이 같은 의미임을 알기 쉬워집니다.

한계는 사전에 없는 말에 약하다는 점입니다. 규칙과 사전을 기반으로 동작하기 때문에 새로 생긴 말이 들어오면 엉뚱하게 쪼깁니다.

이 프로젝트에서 — KoNLPy의 Okt를 썼습니다(구어체·이모티콘에 강해서 선택). 실제로 신조어 "갯생살기"를 넣으니 갯 / 생 / 살기로 의미 없이 쪼개졌고, 이것이 서브워드로 넘어간 직접적 이유가 됐습니다. Okt는 자바로 만들어져 있어 실행에 JDK 설치가 필요합니다.

OOV (미등록어) (Out-Of-Vocabulary)

모델이 알고 있는 단어 목록에 없는 단어.

모델은 미리 정해둔 단어 목록(vocab)에 있는 것만 알아봅니다. 목록에 없는 단어가 들어오면 "모르는 것"(UNK) 하나로 뭉뚱그려 처리되어 정보가 통째로 날아갑니다. 신조어·오타·외래어가 많은 실제 대화에서는 이게 꽤 자주 일어납니다.

이 프로젝트에서 — 형태소 분석 방식의 이 약점이 서브워드(BPE)로 갈아탄 결정적 이유였습니다.

서브워드 / BPE (Subword / Byte Pair Encoding)

단어를 더 작은 "자주 붙어다니는 조각"으로 쪼개는 방식.

핵심 아이디어는 "자주 같이 나오는 글자쌍을 계속 합쳐나가기"입니다. 데이터를 보면서 "ㅏ+ㅑ"가 자주 붙으면 "ㅑ"로 합치고, "ㅑ+랑"이 자주 붙으면 "ㅑ랑"으로 합치는 식으로 정해진 개수가 될 때까지 반복합니다.

이러면 자주 쓰는 말은 통째로 한 조각이 되고, 드문 말은 여러 조각으로 쪼개집니다. 결과적으로 어떤 문자열이든 표현할 수 있어 OOV가 사실상 사라집니다. 요즘 나오는 거의 모든 언어 모델(GPT, BERT, BART 등)이 이 방식을 씁니다.

이 프로젝트에서 — `SentencePiece`로 우리 데이터에서 직접 학습해 vocab 8,000개를 만들었습니다. 아까 Okt가 실패한 "갓생살기"도 BPE는 갓 / 생 / 살 / 기 조합으로 무리 없이 표현합니다.

어휘 사전 (Vocabulary, vocab)

모델이 아는 토큰 전체 목록. 각 토큰에 번호가 매겨져 있다.

{0: "<pad>", 1: "<unk>", ..., 523: "사랑", 524: "하세요"} 같은 번호표입니다. 모델은 글자가 아니라 이 번호로 계산합니다. vocab이 너무 크면 외울 게 많아 비효율적이고, 너무 작으면 문장이 잘게 쪼개져 길어집니다.

이 프로젝트에서 — 우리가 직접 학습한 BPE는 8,000개, 가져다 쓴 KoBART는 30,000개입니다. 8,000을 고른 이유는 학습 말뭉치가 1.9만 줄로 작아서, 3만 이상으로 잡으면 거의 안 쓰이는 조각만 늘기 때문입니다.

특수 토큰 (Special Tokens (BOS, EOS, PAD, UNK))

실제 단어는 아니지만 모델 동작에 꼭 필요한 표시용 토큰.

- BOS (Begin Of Sentence) — 문장 시작 표시. "이제부터 만들어라"
- EOS (End Of Sentence) — 문장 끝 표시. "여기서 멈춰라"
- PAD (Padding) — 길이를 맞추려고 채우는 빈칸
- UNK (Unknown) — vocab에 없는 단어 자리

특히 EOS가 중요합니다. 모델은 한 글자씩 순서대로 만들어내는데, EOS를 내야 "다 썼다"고 판단하고 멈춥니다. EOS를 배우지 못한 모델은 멈출 줄 모르고 계속 지껄입니다.

이 프로젝트에서 — 이 프로젝트에서 가장 큰 사고가 바로 EOS에서 났습니다. KoBART 학습 데이터에 EOS를 빠뜨려서 모델이 문장 끝내는 법을 못 배웠고, 응답의 68%가 중간에 잘렸습니다. EOS를 붙여 다시 학습하자 성능 점수가 2배 이상 올랐습니다. 자세한 내용은 10장에 있습니다.

4. 임베딩 — 단어를 숫자로 바꾸기

토큰화로 문장을 조각내고 번호를 붙였습니다. 그런데 번호 자체에는 의미가 없습니다. "사랑"이 523번이고 "미움"이 524번이라고 해서 둘이 비슷한 뜻인 건 아니니까요. 이 문제를 푸는 것이 임베딩입니다.

원-핫 인코딩 (One-Hot Encoding)

단어 하나를 "그 자리만 1, 나머지는 전부 0"인 긴 벡터로 표현하는 방식.

vocab이 8,000개라면 각 단어는 길이 8,000짜리 벡터가 되고, 그중 자기 자리 하나만 1입니다. 단순하지만 두 가지 치명적 문제가 있습니다.

- ① 너무 낭비입니다. 8,000칸 중 7,999칸이 0입니다.
- ② 의미를 전혀 담지 못합니다. "사랑"과 "애정"을 비교해도, "사랑"과 "컴퓨터"를 비교해도 거리가 똑같습니다. 어느 두 단어든 서로 완전히 남남으로 취급됩니다.

이 프로젝트에서 — 직접 쓰지는 않았지만, 왜 임베딩이 필요한가를 이해하는 출발점입니다.

임베딩 / 분산 표현 (Embedding / Distributed Representation)

단어를 "의미가 담긴 짧은 숫자 목록"으로 바꾸는 것.

8,000칸짜리 희소한 벡터 대신, 예를 들어 100칸짜리 뾰뾰한 숫자 목록으로 표현합니다. '사랑' = [0.21, -0.85, 0.33, ...] 같은 식입니다.

핵심은 이 숫자들이 학습으로 정해진다는 점입니다. 비슷한 맥락에서 쓰이는 단어들은 비슷한 숫자 목록을 갖게 됩니다. 그래서 "사랑"과 "애정"은 가까운 위치에, "사랑"과 "컴퓨터"는 먼 위치에 놓입니다. 즉 의미가 공간상의 위치로 표현되는 것입니다.

원리는 "비슷한 이웃과 어울리는 단어는 비슷한 뜻이다"라는 가정입니다. "나는 ○○을 먹었다"의 ○○에 들어갈 수 있는 단어들은 서로 비슷할 가능성이 높습니다.

이 프로젝트에서 — `gensim`으로 100차원 임베딩을 직접 학습했습니다. 학습된 단어는 6,048개입니다.

임베딩 차원 (Embedding Dimension)

단어 하나를 표현하는 숫자의 개수.

100차원이면 단어 하나당 숫자 100개입니다. 크면 표현력이 늘지만 학습할 것이 많아져 데이터가 적을 때는 오히려 과적합합니다. 작으면 서로 다른 단어를 구별할 여유가 부족합니다. 보통 데이터 규모에 맞춰 정합니다.

이 프로젝트에서 — 100차원을 골랐습니다. 학습 말뭉치가 1.9만 문장으로 작아서 300차원 같은 큰 값은 과적합 위험이 있다고 판단했습니다.

Word2Vec / skip-gram / CBOW (Word2Vec)

주변 단어 관계로 임베딩을 학습하는 대표적인 방법.

두 가지 학습 방식이 있습니다.

- CBOW — 주변 단어들을 보고 가운데 단어를 맞춥니다. ("나는 ○○을 먹었다" → ○○?)
- skip-gram — 반대로 가운데 단어 하나로 주변 단어들을 맞춥니다. ("사과" → 나는/을/먹었다)

skip-gram은 단어 하나로 여러 번 학습하므로 드물게 나오는 단어를 더 잘 배웁니다. CBOW는 빠르지만 자주 나오는 단어에 유리합니다.

이 프로젝트에서 — skip-gram(sg=1)을 골랐습니다. 챗봇 대화는 구어체 표현이 다양하고 드문 단어가 많아서 skip-gram이 유리하다고 판단했습니다.

코사인 유사도 (Cosine Similarity)

두 벡터가 같은 방향을 가리키는 정도. 1에 가까울수록 비슷하다.

두 단어 벡터 사이의 각도를 봅니다. 완전히 같은 방향이면 1, 직각이면 0, 반대 방향이면 -1입니다. 거리 대신 각도를 쓰는 이유는, 단어 벡터의 길이보다 방향이 의미를 담고 있기 때문입니다. 임베딩이 잘 학습됐는지 확인하는 가장 흔한 방법입니다.

이 프로젝트에서 — 학습된 임베딩을 이렇게 검증했습니다 — "이별"의 유사 단어를 찾아보니 폭풍(0.832) / 환승(0.802) / 만남(0.798) / 재회(0.797)가 나왔습니다. 이별과 관련된 말들이 실제로 가까이 모여 있다는 뜻이므로, 임베딩이 의미를 제대로 잡았다고 판단했습니다.

사전학습 임베딩으로 초기화 (Pre-trained Embedding Initialization)

모델을 만들 때 임베딩을 무작위로 시작하지 않고, 미리 학습해둔 값으로 시작하는 것.

모델의 임베딩은 보통 무작위 숫자로 시작해서 학습하며 다듬어집니다. 그런데 이미 좋은 임베딩이 있다면 거기서부터 시작하면 더 빨리 좋아지지 않을까? 하는 아이디어입니다.

이 프로젝트에서 — 실제로 실험했습니다(06번 A1). 결과는 미묘했습니다 — val loss는 확실히 좋아졌지만(5.9692 → 5.8180) 실제 생성 품질(BLEU)은 오히려 낮았습니다(2.21 → 2.50이지만 다른 설정보다 낮음). 게다가 단위가 달라서(Word2Vec은 형태소, 모델은 BPE) 8,000개 중 53.4%만 연결할 수 있었습니다. "loss가 좋아졌다고 결과물이 좋아지는 건 아니다"를 보여준 사례입니다.

가중치 공유 (Weight Tying)

두 군데서 쓰이는 숫자 뭉치를 하나로 합쳐서 같이 쓰는 것.

번역 모델은 보통 "입력 단어 → 숫자"(임베딩)와 "숫자 → 출력 단어"(출력층) 두 개의 큰 표를 갖습니다. 그런데 입력과 출력이 같은 언어, 같은 단어 목록이라면 굳이 두 개를 따로 둘 이유가 없습니다. 하나로 합치면 기억할 숫자가 절반으로 줄어 작은 데이터에서 과적합 위험이 낮아집니다.

이 프로젝트에서 — 적용했습니다. Q와 A가 둘 다 한국어이고 같은 BPE 사전을 쓰기 때문에 가능했습니다. 수업에서 다룬 번역(독일어→영어)에서는 두 언어의 사전이 달라서 쓸 수 없는 기법입니다.

5. 모델 구조 — Transformer 이해하기

이제 숫자로 바뀐 문장을 실제로 처리할 "모델"을 만들 차례입니다. 요즘 언어 모델은 거의 전부 Transformer라는 구조를 씁니다.

신경망 / 파라미터 (Neural Network / Parameters)

숫자를 입력받아 계산해서 숫자를 내놓는 함수 덩어리. 파라미터는 그 계산에 쓰이는 조절 가능한 숫자들. 신경망은 결국 아주 복잡한 함수입니다. 입력 숫자에 어떤 수를 곱하고 더해서 출력을 만드는데, 이때 "곱하는 수"와 "더하는 수"가 파라미터입니다. 학습이란 이 파라미터들을 조금씩 고쳐서 원하는 출력이 나오게 만드는 과정입니다.

파라미터가 많을수록 표현력이 좋지만, 그만큼 학습에 많은 데이터가 필요합니다. 데이터는 적는데 파라미터만 많으면 외워버립니다(과적합).

이 프로젝트에서 — 직접 만든 베이스라인은 6,010,688개(약 600만), 가져다 쓴 KoBART는 123,859,968개(약 1.24억)로 약 21배 차이입니다. 우리 데이터는 9,400쌍뿐이라 베이스라인을 일부러 작게 만들었습니다.

어텐션 / 셀프 어텐션 (Attention / Self-Attention)

문장 안의 각 단어가 다른 단어들 중 "어디를 얼마나 봐야 할지" 스스로 정하는 방식. "그 남자는 사과를 먹었는데 그것은 맛있었다"에서 "그것"이 무엇을 가리킬까요? 사과입니다. 어텐션은 바로 이런 연결을 계산으로 찾아냅니다. "그것"을 처리할 때 문장의 모든 단어에 점수를 매기고, "사과"에 높은 점수를 줘서 거기에 집중하는 식입니다.

이전 방식(RNN)은 문장을 앞에서부터 한 단어씩 순서대로 읽어서 먼 거리의 관계를 놓치기 쉬웠습니다. 어텐션은 모든 단어를 한 번에 보고 거리와 상관없이 직접 연결하기 때문에 훨씬 강력합니다.

"셀프"는 같은 문장 안에서 서로를 본다는 뜻입니다.

이 프로젝트에서 — Transformer의 핵심 부품으로 인코더·디코더 양쪽에 들어 있습니다. 단점은 문장이 길어질수록 계산량이 제곱으로 늘어난다는 것이라, 입력 길이를 16으로 제한했습니다.

크로스 어텐션 (Cross-Attention)

답을 만들 때 질문 쪽을 참고하도록 연결해주는 어텐션. 셀프 어텐션이 "내 문장 안에서 서로 보기"라면, 크로스 어텐션은 "답을 쓰면서 질문을 결눈질하기"입니다. 답변의 각 단어를 만들 때마다 질문의 어느 부분이 중요한지 보고 반영합니다. 이것이 있어야 "입력에 맞는 답"이 나옵니다 — 조건부 생성의 핵심 장치입니다.

이 프로젝트에서 — 디코더가 인코더의 결과를 참고하는 통로입니다. 이 구조 덕분에 "배터리 겨우 15%야"라는 질문에 "충전하고 만나요."처럼 질문 내용에 반응하는 답이 나옵니다.

인코더 / 디코더 (Encoder / Decoder)

입력을 읽어 요약하는 쪽이 인코더, 그 요약을 보고 출력을 만드는 쪽이 디코더.

통역사에 비유하면 이렇습니다.

- 인코더 = 한국어를 끝까지 듣고 이해하는 단계 (문장 전체를 한꺼번에 봅니다)
- 디코더 = 이해한 내용을 바탕으로 영어를 한 단어씩 말하는 단계

디코더가 한 번에 한 단어씩 만드는 이유는, 다음 단어를 정하려면 지금까지 만든 단어들을 알아야 하기 때문입니다.

이 프로젝트에서 — 인코더 3층 + 디코더 3층으로 구성했습니다. 원 논문은 각 6층이지만 데이터가 작아 절반으로 줄였습니다.

인코더-디코더 vs 디코더 전용 (Encoder-Decoder vs Decoder-only)

입력과 출력을 구조적으로 분리할 것인가, 하나로 이어붙일 것인가.

- 인코더-디코더 (BART, T5, 번역 모델) — 입력용 부품과 출력용 부품이 따로 있습니다. 입력과 출력의 역할 구분이 구조에 새겨져 있습니다.
- 디코더 전용 (GPT 계열) — "질문 + 답변"을 한 줄로 이어붙이고 다음 단어 맞히기만 반복합니다. 단순하고 확장성이 좋아 요즘 대형 모델은 대부분 이 방식입니다. 대신 어디까지가 질문이고 어디부터가 답인지를 모델이 데이터만 보고 스스로 배워야 합니다.

이 프로젝트에서 — 인코더-디코더를 선택했습니다. 이유는 Q와 A의 역할이 뚜렷이 다르고, 데이터가 1만 건 정도로 적어서 "경계를 스스로 배우는" 부담을 지우기보다 구조가 미리 알려주는 편이 유리하다고 판단했기 때문입니다.

포지셔널 인코딩 (Positional Encoding)

단어의 "순서" 정보를 따로 넣어주는 장치.

어텐션은 모든 단어를 한꺼번에 보기 때문에, 그냥 두면 순서를 전혀 모릅니다. "개가 사람을 물었다"와 "사람이 개를 물었다"를 구별하지 못합니다. 그래서 각 위치마다 고유한 숫자 패턴을 만들어 단어 벡터에 더해줍니다. "이건 3번째 단어야"라는 표시를 붙이는 셈입니다.

이 프로젝트에서 — 사인·코사인 함수로 만드는 고전적인 방식을 썼습니다(원 논문과 동일).

마스킹 (Masking)

보면 안 되는 부분을 가려서 계산에서 제외하는 것.

두 종류를 씁니다.

- 패딩 마스크 — 길이 맞추려고 넣은 빈칸(PAD)을 진짜 단어로 착각하지 않게 가립니다.
- 인과 마스크 (causal mask) — 디코더가 답을 만들 때 아직 안 만든 뒷부분을 미리 훑쳐보지 못하게 가립니다. 이걸 안 하면 학습 때는 정답을 보고 맞히니 잘하는데, 실제로 쓸 때는 볼 게 없어서 망합니다.

이 프로젝트에서 — 두 마스크 모두 구현해 적용했습니다.

6. 학습 — 모델을 훈련시키기

손실 함수 / loss (Loss Function)

모델의 답이 정답에서 얼마나 틀렸는지를 하나의 숫자로 나타낸 것.

낮을수록 좋습니다. 학습이란 결국 "이 숫자를 낮추는 방향으로 파라미터를 조금씩 고치는" 반복입니다. 생성 문제에서는 보통 교차 엔트로피(Cross Entropy)를 쓰는데, "정답 단어에 얼마나 높은 확률을 줬는가"를 재는 방식입니다. 정답에 100% 확신하면 loss가 0에 가깝고, 엉뚱한 단어에 확신하면 loss가 커집니다.

이 프로젝트에서 — train loss와 val loss를 매 epoch 기록했습니다. 베이스라인은 29.82에서 시작해 117 epoch 만에 val loss 5.7957까지 내려갔고, KoBART는 7 epoch 만에 2.7424에 도달했습니다.

레이블 스무딩 (Label Smoothing)

모델이 정답 하나에 과하게 확신하지 않도록 일부러 살짝 흐리게 만드는 기법.

정답에 "100% 확신해"라고 가르치는 대신 "90% 정도만 확신하고 나머지는 열어둬"라고 가르칩니다. 생성 문제에서는 정답이 여러 개일 수 있으니, 하나에만 확신하는 모델은 매번 똑같은 뻔한 답만 내놓게 됩니다. 적당히 흐리게 하면 표현이 다양해집니다.

이 프로젝트에서 — 0.1을 적용했습니다. 짧은 챗봇 응답은 비슷한 표현이 많아 과확신 위험이 특히 큼니다.

옵티마이저 / 학습률 (Optimizer / Learning Rate)

loss를 낮추는 방향으로 파라미터를 고치는 방법(옵티마이저)과, 한 번에 얼마나 크게 고칠지(학습률).

안개 낀 산에서 가장 낮은 곳을 찾아 내려가는 상황을 상상해 보세요. 옵티마이저는 내려가는 전략이고, 학습률은 보폭입니다.

보폭이 너무 크면 골짜기를 건너뛰어 헤매고, 너무 작으면 영원히 도착하지 못합니다. 학습률은 딥러닝에서 가장 중요한 하이퍼파라미터로 꼽힙니다.

이 프로젝트에서 — Adam 옵티마이저를 썼고, 학습률을 바꿔가며 실험했습니다. 베이스라인에서 $lr=1e-4(5.9692)$ 보다 $lr=3e-4(5.7364)$ 가 더 좋았지만, 생성문에 불안정한 출력도 늘었습니다 — "빠르면 불안정해진다"는 트레이드오프를 실측으로 확인한 셈입니다.

에폭 / 배치 크기 (Epoch / Batch Size)

학습 데이터를 통째로 한 바퀴 도는 것이 1 epoch, 한 번에 묶어 처리하는 샘플 수가 배치 크기.

9,400개 데이터를 64개씩 묶으면 147묶음이고, 이 147묶음을 다 처리하면 1 epoch입니다. 보통 수십~수백 epoch을 둡니다.

배치가 너무 작으면 방향이 들쭉날쭉해지고, 너무 크면 파라미터를 고치는 횟수가 줄어 학습이 더뎡니다.

이 프로젝트에서 — 배치 64(베이스라인) / 16(KoBART)를 썼습니다. KoBART가 훨씬 커서 메모리 때문에 줄었습니다. epoch 수는 이 프로젝트에서 가장 극적인 효과를 낸 설정이었습니다 — 30에서 117로 늘렸더니 반복 오류가 8.3%에서 1.4%로 줄었습니다.

조기 종료 (Early Stopping)

val loss가 더 이상 좋아지지 않으면 학습을 알아서 멈추는 장치.

"몇 epoch 연속으로 나아지지 않으면 그만"이라는 규칙입니다. 이 "몇 번"을 patience라고 합니다. 과적합이 시작된 뒤에도 계속 학습하면 시간만 버리고 성능은 나빠지므로, val loss가 가장 좋았던 시점의 모델을 저장해두고 거기서 멈춥니다.

이 프로젝트에서 — KoBART는 7 epoch에서 조기 종료됐습니다(5 epoch부터 val loss 반등). 반대로 03번 베이스라인은 조기 종료가 걸리지 않고 30 epoch을 다 써버렸는데, 이건 "아직 더 배울 수 있었다"는 신호였고 실제로 04번에서 117 epoch까지 늘리자 크게 좋아졌습니다.

티쳐 포싱 (Teacher Forcing)

학습할 때는 모델이 틀려도 정답을 다음 입력으로 넣어주는 방식.

답을 한 단어씩 만드는 모델을 학습시킬 때, 모델이 3번째 단어를 틀렸다고 해서 그 틀린 단어를 4번째 입력으로 주면 뒤가 전부 엉망이 됩니다. 그래서 학습 중에는 항상 정답을 넣어줍니다.

문제는 실제로 쓸 때는 정답이 없다는 것입니다. 자기가 만든 단어를 스스로 다시 입력해야 합니다. 이 학습 때와 실전의 차이를 exposure bias라고 부릅니다.

이 프로젝트에서 — 이것이 "val loss만 믿으면 안 되는" 이유입니다. val loss는 티쳐 포싱 상태에서 잦 값이라 실제 생성 성능과 다를 수 있습니다. 그래서 07번에서 실제로 문장을 생성시켜 따로 채점했습니다.

체크포인트 (Checkpoint)

학습 도중 모델 상태를 파일로 저장해둔 것.

학습한 파라미터를 파일로 남겨두면 나중에 다시 불러 쓸 수 있습니다. 보통 val loss가 가장 좋았던 시점의 것을 저장합니다.

이 프로젝트에서 — `checkpoints/` 폴더에 9개 모델을 저장했습니다. 07번 평가는 학습을 다시 하지 않고 이 파일들을 불러와서 채점만 했습니다.

통제 실험 / 대조군 (Controlled Experiment / Control Group)

한 번에 딱 하나만 바꿔서, 변화의 원인을 특정할 수 있게 만드는 실험 설계.

두 가지를 동시에 바꾸고 결과가 좋아지면, 둘 중 무엇 때문인지 알 수 없습니다. 그래서 나머지를 전부 고정하고 하나만 바꿉니다. 이때 바뀌지 않아야 하는 쪽(대조군)이 정말 안 바뀌는지 확인하면 실험이 제대로 됐다는 증거가 됩니다.

이 프로젝트에서 — 이 프로젝트의 핵심 방법론입니다. 03→04에서 epoch 상한만 바꿔 "반복의 원인은 학습 부족"을 증명했고, EOS 수정 때도 EOS만 바꿨습니다. 그 결과 베이스라인과 KE-T5(대조군 2개)의 점수가 소수점까지 하나도 변하지 않아 원인 규명이 정확했음이 확인됐습니다.

7. 전이학습 — 남이 배운 걸 물려받기

이 프로젝트의 핵심 주제입니다. 우리 데이터는 9,400쌍뿐인데, 그걸로 한국어를 처음부터 가르치는 건 무리입니다. 이미 한국어를 아는 모델을 데려와서 우리 일만 가르치면 어떨까요?

사전학습 (Pre-training)

대량의 일반 텍스트로 언어 자체를 미리 배워두는 단계.

위키백과·뉴스·책 등 수억~수십억 단어를 읽히면서 "한국어란 이런 것이다"를 배우게 합니다. 이 단계에서는 특정 과제를 가르치지 않고, 문장의 일부를 가리고 맞추게 하는 식으로 언어의 일반적인 패턴만 익히게 합니다.

어마어마한 컴퓨터 자원이 들기 때문에 보통 개인이 직접 하지 않고, 공개된 것을 가져다 씁니다.

이 프로젝트에서 — 직접 하지 않았습니다. `gogamza/kobart-base-v2`(KoBART)와 `KETI-AIR/ke-t5-base`(KE-T5)를 Hugging Face에서 내려받아 썼습니다.

파인튜닝 (Fine-tuning)

사전학습된 모델을 내 과제 데이터로 조금 더 학습시켜 다듬는 것.

한국어를 이미 아는 사람에게 "이제 챗봇 말투로 답하는 법"만 가르치는 것과 같습니다. 밑바닥부터 한국어를 가르치는 것보다 훨씬 적은 데이터와 시간으로 됩니다.

주의할 점은 학습률을 작게 해야 한다는 것입니다. 크게 하면 애써 배운 지식을 덮어서 버립니다. 보통 사전학습 때보다 10~100배 작은 값을 씁니다.

이 프로젝트에서 — $lr=5e-5$ 로 7 epoch만 학습시켰습니다. 결과는 극적이었습니다 — 베이스라인이 117 epoch 걸려 도달한 수준(val loss 5.80)을 KoBART는 7 epoch 만에 크게 앞질렀습니다(2.74). 전이학습의 효과를 수치로 증명한 핵심 결과입니다.

전이학습 (Transfer Learning)

한 곳에서 배운 지식을 다른 과제로 옮겨 쓰는 것. 사전학습 + 파인튜닝의 전체 흐름.

사람도 그렇습니다. 피아노를 칠 줄 아는 사람이 기타를 배우면 악보를 못 읽는 사람보다 빠릅니다. 음악의 일반 지식이 옮겨가기 때문입니다.

왜 잘 될까요? 사전학습 모델은 이미 문법·어휘·문맥을 알고 있어서, 우리가 가르칠 것은 "이 과제에서는 어떤 식으로 답하면 되는가"뿐이기 때문입니다.

이 프로젝트에서 — 최종 결과로 전이학습이 베이스라인을 BLEU 기준 62% 앞섰습니다(7.90 vs 4.87). 다양성 지표도 68% 높았고, 학습 비용은 오히려 훨씬 적었습니다.

KoBART / KE-T5 ()

한국어로 사전학습된 인코더-디코더 모델 두 종류.

· KoBART — BART 구조. 문장을 망가뜨린 뒤 원래대로 복원하는 방식으로 사전학습됩니다. 복원 자체가 "문장 만들기"라서 생성 과제에 잘 맞습니다.

· KE-T5 — T5 구조. 문장의 일정 구간을 통째로 가리고 채우는 방식(span corruption)으로 학습됩니다. 범용성이 좋지만 생성 과제에 적응하는 데 시간이 더 걸립니다.

이 프로젝트에서 — 같은 조건(7 epoch)에서 KoBART가 압도적으로 좋았습니다(val loss 2.74 vs 4.64). KE-T5는 파라미터가 2배(2.47억)나 많은데도 거의 모든 질문에 "울어도 괜찮아요."만 답하는 모드 붕괴 상태였습니다. 모델이 크다고 좋은 게 아니라, 사전학습 방식과 과제의 궁합, 그리고 주어진 학습 예산이 함께 작용한다는 교훈을 얻었습니다.

중요 — 토큰라이저는 반드시 그 모델 것을 써야 합니다.

사전학습 모델의 임베딩은 그 모델 토큰라이저의 번호표에 맞춰 학습되어 있습니다. 523번이 "사랑"인 줄 알고 학습했는데 우리가 다른 토큰라이저를 쓰면 523번이 "컴퓨터"가 되어버립니다. 그러면 애써 사전학습한 지식이 전부 무의미해집니다.

그래서 이 프로젝트는 토큰라이저를 두 개 병행했습니다 — 직접 만든 베이스라인에는 우리 BPE(8,000), 전이학습 모델에는 각 모델 고유 토큰라이저(KoBART 30,000).

8. 디코딩 — 문장을 실제로 만들어내는 방법

학습이 끝난 모델은 매 순간 "다음에 올 단어별 확률"을 내놓습니다. 그 확률에서 실제로 어떤 단어를 고를 것인가가 디코딩 전략입니다. 중요한 점은 이게 모델과 별개의 선택지라는 것입니다 — 같은 모델이라도 뽑는 방식만 바꾸면 결과가 확 달라집니다.

탐욕적 디코딩 (Greedy Decoding)

매 순간 확률 1등 단어만 고르는 가장 단순한 방법.

빠르고 결과가 항상 같습니다. 하지만 근시안적입니다. 매 걸음 눈앞에서 제일 좋은 길만 고르다 보면 전체적으로는 나쁜 길로 빠질 수 있습니다. 특히 같은 말을 반복하는 루프에 잘 갇힙니다.

이 프로젝트에서 — 전체 평가에서 beam search보다 확실히 나빴습니다(베이스라인 BLEU 3.05 vs 4.87). 반복률도 0.121로 가장 높았습니다.

빔 서치 (Beam Search)

후보 문장 여러 개를 동시에 끌고 가다가 마지막에 제일 좋은 걸 고르는 방법.

greedy가 길 하나만 따라간다면, beam search는 유망한 길 k개를 동시에 탐색합니다. 매 단계마다 각 후보를 이어보고, 전체 점수가 높은 상위 k개만 남깁니다. 끝까지 간 뒤 최종 점수가 가장 높은 문장을 고릅니다.

greedy보다 전체적으로 그럴듯한 문장을 찾지만, 계산량이 k배로 늘고 다양성은 오히려 줄어듭니다 (안전하고 뻔한 답으로 수렴하는 경향).

이 프로젝트에서 — beam=5를 썼습니다. 효과가 컸습니다 — 베이스라인 BLEU를 3.05 → 4.87로 60% 끌어올렸고, 반복률도 0.121 → 0.014로 거의 없었습니다. 모델을 바꾸지 않고 얻은 개선이라는 점이 중요합니다.

길이 페널티 (Length Penalty)

빔 서치가 짧은 문장만 좋아하는 편향을 보정하는 장치.

문장 점수는 단어 확률을 계속 곱해서(로그로는 더해서) 만드는데, 단어가 많아질수록 점수가 깎입니다. 그래서 그냥 두면 무조건 짧은 문장이 유리합니다. 길이로 나눠주는 보정을 넣어 공평하게 만듭니다.

이 프로젝트에서 — 0.7을 적용했습니다(GNMT 논문 방식).

샘플링 / temperature / top-k / top-p (Sampling)

확률에 따라 무작위로 뽑되, 이상한 단어는 미리 걸러내는 방법.

항상 1등만 뽑으면 매번 같은 답이 나옵니다. 샘플링은 주사위를 굴러 뽑아서 다양성을 만듭니다. 다만 그냥 굴리면 말도 안 되는 단어가 튀어나올 수 있어 필터를 씁니다.

- temperature — 확률 분포의 뽀족함 조절. 낮으면(0.8) 안전하게, 높으면(1.2) 과감하게.
- top-k — 확률 상위 k개 안에서만 뽑기 (k=50이면 51등 이하는 후보에서 제외)
- top-p (nucleus) — 확률을 높은 순으로 더해 p%가 될 때까지만 후보로 (p=0.9)

이 프로젝트에서 — temperature 0.8 / top-k 50 / top-p 0.9를 썼습니다. 예상대로 다양성은 가장 높았지만(Distinct-2 0.563) 정답 일치도는 가장 낮았습니다(BLEU 3.06). "다양성과 정확도는 상충한다"를 수치로 보여준 사례입니다.

n-gram 반복 억제 (no_repeat_ngram)

같은 n개 단어 묶음이 두 번 나오지 않게 강제하는 옵션.

`no\_repeat\_ngram\_size=3`이면 3단어 묶음이 이미 나왔을 경우 다시 못 쓰게 막습니다. 반복을 없애는 데 즉효라 실무에서 흔히 씁니다.

이 프로젝트에서 — 이 프로젝트에서 중요한 교훈을 준 설정입니다. 처음에는 "이게 반복을 없애는 데 결정적"이라고 결론 내렸습니다. 그런데 나중에 반복의 진짜 원인(EOS 누락)을 찾아 고치자, 반복이 저절로 사라져서 이 옵션은 사실상 무효가 되었습니다(추가 이득 0.01).

즉 이것은 증상을 덮는 임시방편이었습니다. 원인을 고치는 것이 옳은 해법이었습니다.

9. 평가지표 — 생성된 문장을 채점하기

생성 문제의 가장 어려운 부분입니다. 정답이 하나가 아닌데 어떻게 점수를 매길까요? 완벽한 방법은 없고, 서로 다른 약점을 가진 여러 지표를 함께 봅니다.

n-gram ()

연속된 n개의 토큰 묶음.

"감기 조심 하세요"라면

- 1-gram: 감기 / 조심 / 하세요
- 2-gram: (감기,조심) / (조심,하세요)
- 3-gram: (감기,조심,하세요)

n이 클수록 어순까지 따지게 됩니다. 여러 지표의 기본 재료입니다.

이 프로젝트에서 — 형태소 단위로 n-gram을 만들어 BLEU와 Distinct 계산에 썼습니다.

정밀도 / 재현율 / F1 (Precision / Recall / F1)

맞힌 것 중 진짜 비율(정밀도), 맞혀야 할 것 중 맞힌 비율(재현율), 둘의 조화평균(F1).

- 정밀도 = 내가 쓴 단어 중 정답에도 있는 비율 → 헛소리를 안 했는가
- 재현율 = 정답 단어 중 내가 쓴 비율 → 빠뜨리지 않았는가

둘은 상충합니다. 아무 말이나 잔뜩 쓰면 재현율은 오르지만 정밀도는 떨어집니다. 그래서 균형을 보는 F1을 함께 씁니다.

이 프로젝트에서 — BLEU는 정밀도 기반, ROUGE-L은 F1 기반이라 둘을 같이 보면 서로를 보완합니다.

BLEU (Bilingual Evaluation Understudy)

생성문의 n-gram이 정답에 얼마나 들어있는지 보는 정밀도 중심 점수 (0~100).

원래 기계번역 채점용으로 만들어졌습니다. 1~4gram이 정답과 겹치는 비율을 종합하고, 너무 짧게 써서 점수를 따는 걸 막는 짧은 문장 패널티를 곱합니다.

한계가 뚜렷합니다 — 표현만 보고 의미는 안 봅니다. "당연히 가능하죠"와 "가능합니다!"는 뜻이 같은데 겹치는 형태소가 없어 0점이 됩니다.

이 프로젝트에서 — 최고 7.90(KoBART B2)이 나왔습니다. 절대값이 낮아 보이지만, 정답이 하나뿐인 대화 데이터에서는 자연스러운 수준이며 모델 간 상대 비교가 목적입니다.

ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation)

정답과 생성문의 "가장 긴 공통 순서"를 기준으로 재는 점수.

LCS(최장 공통 부분수열)를 씁니다. 순서는 지키되 중간에 다른 단어가 끼어도 괜찮게 세는 방식입니다. "감기 정말 조심하세요"와 "감기 조심하세요"는 "감기...조심하세요"가 공통이라 높은 점수를 받습니다. BLEU보다 유연하고 요약 평가에 많이 씁니다.

이 프로젝트에서 — 최고 25.14(KoBART B2)입니다. BLEU와 순위가 대체로 같아 서로를 뒷받침했습니다.

Distinct-n ()

생성한 응답 전체에서 서로 다른 n-gram이 차지하는 비율. 다양성 지표.

정답과 비교하지 않는다는 점이 특별합니다. 모델이 만든 답변들끼리 비교해서, "1,175개 질문에 얼마나 서로 다른 답을 했는가"를 봅니다.

이게 없으면 모든 질문에 똑같이 답하는 모델을 잡아낼 수 없습니다. BLEU/ROUGE는 우연히 잘 맞는 무난한 답 하나만 반복해도 어느 정도 점수가 나오기 때문입니다.

이 프로젝트에서 — 이 지표가 두 가지 중대한 문제를 잡아냈습니다.

① baseline-30ep의 Distinct-1이 0.027 — 사실상 모든 질문에 비슷한 답을 하고 있었습니다.

② KE-T5의 Distinct-1이 0.002 — 거의 한 문장만 반복하는 완전한 모드 붕괴였습니다. 이 모델은 반복·빈 응답 같은 규칙 검사로는 가장 깨끗해 보였기 때문에, Distinct가 없었다면 놓쳤을 것입니다.

빈 응답 비율 / 연속 반복률 ()

아무것도 못 만든 비율, 같은 토큰이 연달아 나온 응답의 비율.

보조 지표입니다. 생성 자체가 실패했거나 "마음을 마음을 마음을..."처럼 망가진 경우를 잡습니다.

이 프로젝트에서 — 빈 응답은 9개 모델 모두 0%였습니다. 연속 반복률은 baseline-30ep가 8.3%로 가장 높았고, epoch을 늘리자 1.4%로 떨어졌습니다.

왜 지표를 4개나 볼까요?

각 지표가 잡아내는 실패가 다르기 때문입니다.

- BLEU → 엉뚱한 말을 하는가
- ROUGE-L → 핵심을 빠뜨렸는가
- Distinct-n → 전부 똑같이 답하는가
- 빈 응답률 → 생성 자체가 실패했는가

하나만 보면 반드시 놓칩니다. 실제로 KE-T5는 BLEU만 봤다면 "그냥 성능이 낮은 모델"로 넘어갔겠지만, Distinct를 함께 봤기에 모드 붕괴라는 진단이 가능했습니다.

한국어를 채점할 때의 추가 문제

BLEU/ROUGE는 영어처럼 띄어쓰기가 곧 단어 경계인 언어를 가정합니다. 한국어에 그대로 쓰면 곤란합니다.

채점 단위	문제	예
어절 (띄어쓰기)	조사·띄어쓰기 차이로 과도하게 엄격	"충전하세요" vs "충전 하세요" → 0점
음절 (글자)	한 글자만 겹쳐도 점수 → 과도하게 관대	"사과" vs "사람" → 부분 점수
형태소	의미 단위에 가까운 절충점	둘 다 "충전 / 하세요"로 정규화

표 9-1. 이 프로젝트는 형태소(Okt) 단위로 채점했다. 02번에서 "형태소가 절충점"이라 결론낸 원칙을 평가에도 일관되게 적용한 것이다.

10. 오류 분석 — 왜 틀렸는지 알아내기

점수가 나왔다고 끝이 아닙니다. 왜 그 점수가 나왔는지, 무엇을 고쳐야 하는지를 알아야 다음이 있습니다.

오류 분석 (Error Analysis)

모델이 틀린 사례를 모아 유형별로 분류하고 원인을 찾는 작업.

"성능이 낮다"는 정보로는 아무것도 못 합니다. "어떤 종류의 실수를 얼마나 하는가"를 알아야 다음에 뭘 고칠지 정할 수 있습니다.

요령은 자동으로 셀 수 있는 것과 사람이 읽어야 하는 것을 분리하는 것입니다. 섞으면 "대충 이런 것 같다"는 인상비평이 됩니다.

이 프로젝트에서 — 두 층으로 나눴습니다. A층(자동) — 빈 응답·반복·질문 복사처럼 규칙으로 판정 가능한 것. B층(수동) — 정답과 안 겹치는 사례를 직접 20건 읽고 분류.

모드 붕괴 (Mode Collapse)

모델이 입력과 상관없이 거의 항상 같은 출력만 내놓는 현상.

모델이 "이렇게 답하면 대충 중간은 간다"는 안전한 답 하나를 찾아내고 거기에 갇히는 것입니다. 학습이 충분히 되지 않았거나 데이터가 부족할 때 자주 생깁니다. 개별 문장만 보면 멀쩡해서 알아채기 어렵다는 게 무섭습니다.

이 프로젝트에서 — KE-T5가 정확히 이 상태였습니다. 거의 모든 질문에 "울어도 괜찮아요."만 답했습니다. 문장 자체는 완벽한 한국어라 반복 검사·문법 검사는 전부 통과했고, Distinct-1 0.002라는 숫자로만 드러났습니다.

반복 붕괴 (Repetition / Degeneration)

같은 단어나 구절을 계속 반복하며 문장이 망가지는 현상.

생성 모델의 고질적 문제입니다. 원인이 여러 개일 수 있어 진단이 중요합니다 — 학습 부족일 수도, 디코딩 전략 탓일 수도, 아예 다른 버그일 수도 있습니다.

이 프로젝트에서 — 03번에서 "마음을 마음을 마음을..."처럼 심하게 나타났습니다. 원인을 세 가지로 나눠 가설을 세우고 하나씩 검증했습니다 — (a) 학습 부족 → 맞았음(epoch 확장으로 8.3%→1.4%), (b) 디코딩 → 일부 기여(beam으로 개선), (c) 데이터 한계 → 부분적으로 유효.

실제 사례 — 이 프로젝트에서 찾아낸 진짜 버그

오류 분석이 어떻게 작동하는지 보여주는 가장 좋은 예라서 따로 정리합니다.

상황: 07번 평가 결과, 사전학습 KoBART(BLEU 3.83)가 직접 만든 베이스라인(4.87)에 졌습니다. 그런데 눈으로 본 문장은 분명 KoBART 쪽이 유창했습니다. 뭔가 이상합니다.

갈림길: 여기서 두 가지 해석이 가능합니다.

- (a) 지표가 잘못 재고 있다 — "챗봇은 정답이 여러 개니까 BLEU가 원래 못 잡는 거지"
- (b) 파이프라인에 버그가 있다 — "모델이 실제로 뭔가 잘못하고 있고, 지표가 그걸 정확히 잡은 것"

(a)로 결론 내고 넘어가는 게 쉬운 길이었습니니다. 하지만 (b)부터 검증했고, 실제로 버그가 있었습니다.

단계	무엇을 확인했나	결과
단서 1	생성문 길이와 문장이 제대로 끝나는지 확인	KoBART 응답의 68%가 문장 중간에서 잘림 (정상 종결 32.3%, 베이스라인은 99.6%)
단서 2	학습에 들어간 정답 토큰을 직접 출력해 확인	KoBART 토큰라이저가 EOS를 안 붙이고 있었음 → 모델이 "끝내는 법"을 배운 적이 없음

단계	무엇을 확인했나	결과
단서 3	"정답이 길어서 잘린 것 아닌가?" 대안 가설 검증	잘리는 정답은 0.8%뿐 → 가설 기각
결론	원인 확정	베이스라인은 EOS를 직접 붙였는데, 전이학습 쪽에만 빠뜨린 것

표 10-1. 증상 → 직접 원인 → 근본 원인 순으로 좁혀 들어간 과정. 대안 가설을 명시적으로 기각한 것이 중요하다.

수정 결과: EOS 하나를 붙이고 다시 학습시키자 —

모델	BLEU 전 → 후	정상 종결률 전 → 후	평균 길이 전 → 후 (정답 6.46)
KoBART	3.83 → 7.72	32.3% → 98.0%	12.32 → 6.28
베이스라인 (대조군)	4.87 → 4.87	99.6% → 99.6%	5.18 → 5.18
KE-T5 (자연 대조군)	0.09 → 0.09	100% → 100%	4.00 → 4.00

표 10-2. 대조군 두 개가 전혀 변하지 않은 것이 "EOS가 원인이었다"는 결론의 증거다. KE-T5는 토크나이저가 원래 EOS를 붙여서 이번 수정이 아무 효과가 없는 경우였다.

이 사건의 교훈 세 가지

- ① 지표가 직관과 어긋나면 지표를 의심하기 전에 파이프라인을 의심하라. "지표의 한계"라는 편한 설명으로 넘어갔다면 버그를 영영 못 찾았을 것이다.
- ② 증상을 덮지 말고 원인을 찾아라. 반복을 없애려고 넣었던 `no\_repeat\_ngram`은 사실 이 버그의 증상을 가리고 있었다.
- ③ 대조군을 만들어라. 안 변해야 할 것이 정말 안 변하는지 확인하는 순간, 추측이 증거가 된다.

11. 전체 흐름 한눈에 보기

단계	노트북	핵심 용어	이 프로젝트의 선택
데이터 준비	01	EDA, percentile, train/val/test, stratify, 과적합	11,823 → 11,750건 정제, 8:1:1 층화 분할
토큰화	02	토큰, 형태소, 서브워드/BPE, OOV, vocab, 특수토큰	SentencePiece BPE (vocab 8,000)
임베딩	02	분산 표현, Word2Vec, skip-gram, 코사인 유사도	100차원, skip-gram, 6,048단어
모델 설계	03	어텐션, 인코더-디코더, 포지셔널 인코딩, 마스킹, weight tying	인코더3+디코더3, d_model 256, 600만 파라미터
학습	03-04	loss, Adam, 학습률, epoch, 조기 종료, 티쳐 포싱	lr 1e-4, batch 64, 117 epoch (조기 종료)
전이학습	04-06	사전학습, 파인튜닝, 전이학습	KoBART 파인튜닝 7 epoch → val loss 2.74
디코딩	05	greedy, beam search, 샘플링, temperature, top-k/p	beam=5 + length penalty 0.7
평가	07	BLEU, ROUGE-L, Distinct-n, n-gram, 정밀도/재현율	형태소 단위 채점, test 1,175건
오류 분석	08	오류 분석, 모드 붕괴, 반복 붕괴, 통제 실험	EOS 버그 발견 → BLEU 2배 개선

표 11-1. 프로젝트 단계별 용어 지도.

최종 결과

모델	BLEU	ROUGE-L	Distinct-2	학습 epoch
KoBART (전이학습)	7.90	25.14	0.510	7
베이스라인 (직접 구현)	4.87	22.99	0.303	117
KE-T5 (전이학습)	0.09	19.47	0.003	7 (수렴 실패)

표 11-2. test 1,175건 기준 최종 성능.

이 프로젝트가 남긴 교훈

- 전이학습은 실제로 효과적이다. 적은 데이터·적은 학습으로 더 좋은 결과를 냈다(BLEU +62%, epoch은 1/16).
- loss와 생성 품질은 다르다. val loss가 가장 낮은 모델이 BLEU 1등이 아니었다. 티쳐 포싱 때문에 loss는 실제 생성과 차이가 있다.
- 지표 하나로는 판정할 수 없다. KE-T5는 규칙 검사로는 가장 깨끗했지만 Distinct로만 모드 붕괴가 드러났다.
- 가장 큰 개선은 모델 교체가 아니라 버그 수정에서 나왔다. EOS 하나로 BLEU가 2배가 됐다.
- 한 번에 하나만 바꿔라. 그래야 무엇이 효과였는지 말할 수 있다.

12. 용어 빠른 찾기 (가나다순)

용어	영문	한 줄 정의	장
과적합	Overfitting	학습 데이터를 외워서 새 데이터에 오히려 약해지는 현상	2
교차 엔트로피	Cross Entropy	정답에 얼마나 높은 확률을 줬는지 재는 손실 함수	6
디코더	Decoder	요약된 정보로 출력을 한 토크씩 만들어내는 부분	5
레이블 스무딩	Label Smoothing	정답에 과하게 확신하지 않도록 흐리게 만드는 기법	6
마스킹	Masking	빈칸이나 미래 토크를 가려서 계산에서 제외하는 것	5
말뭉치	Corpus	분석·학습에 쓰는 텍스트 데이터 모음	1
모드 붕괴	Mode Collapse	입력과 무관하게 거의 같은 출력만 내놓는 현상	10
백분위수	Percentile	데이터를 줄 세웠을 때 특정 위치의 값	2
빔 서치	Beam Search	후보 문장 여러 개를 동시에 탐색해 최선을 고르는 디코딩	8
사전학습	Pre-training	대량 텍스트로 언어 자체를 미리 학습하는 단계	7
샘플링	Sampling	확률에 따라 무작위로 단어를 뽑는 디코딩	8
서브워드 / BPE	Subword / BPE	자주 붙어다니는 글자 조각 단위로 쪼개는 토큰화	3
셀프 어텐션	Self-Attention	문장 안 단어들이 서로 어디를 볼지 정하는 방식	5
손실 함수	Loss Function	정답에서 얼마나 틀렸는지를 나타내는 숫자	6
어텐션	Attention	중요한 부분에 집중하도록 가중치를 두는 계산	5
에폭	Epoch	학습 데이터를 통째로 한 바퀴 도는 것	6
오류 분석	Error Analysis	틀린 사례를 유형화해 원인을 찾는 작업	10
옵티마이저	Optimizer	loss를 낮추는 방향으로 파라미터를 고치는 방법	6
원-핫 인코딩	One-Hot	한 자리만 1인 긴 벡터로 단어를 표현하는 방식	4
인코더	Encoder	입력을 읽어 압축된 표현으로 만드는 부분	5
임베딩	Embedding	단어를 의미가 담긴 짧은 숫자 목록으로 바꾸는 것	4
자연어처리	NLP	사람의 말과 글을 컴퓨터가 다루게 하는 기술	1
전이학습	Transfer Learning	한 곳에서 배운 지식을 다른 과제로 옮겨 쓰는 것	7
정밀도/재현율	Precision/Recall	헛소리 안 했는가 / 빠뜨리지 않았는가	9
조건부 생성	Conditional Generation	주어진 입력에 맞춰 문장을 만드는 것	1
조기 종료	Early Stopping	val loss가 안 좋아지면 학습을 멈추는 장치	6
층화 추출	Stratified Sampling	종류별 비율을 유지하며 데이터를 나누는 것	2
코사인 유사도	Cosine Similarity	두 벡터가 같은 방향인 정도 (1에 가까울수록 유사)	4
크로스 어텐션	Cross-Attention	출력을 만들며 입력 쪽을 참고하는 어텐션	5
탐욕적 디코딩	Greedy Decoding	매 순간 확률 1등만 고르는 디코딩	8
토큰화	Tokenization	문장을 처리 단위로 쪼개는 작업	3
통제 실험	Controlled Experiment	한 번에 하나만 바꿔 원인을 특정하는 실험 설계	6
티쳐 포싱	Teacher Forcing	학습 중 항상 정답을 다음 입력으로 넣어주는 방식	6
파라미터	Parameters	모델이 학습으로 조절하는 숫자들	5
파인튜닝	Fine-tuning	사전학습 모델을 내 데이터로 다듬는 것	7
포지셔널 인코딩	Positional Encoding	단어의 순서 정보를 넣어주는 장치	5
학습률	Learning Rate	파라미터를 한 번에 얼마나 크게 고칠지	6

용어	영문	한 줄 정의	장
형태소	Morpheme	뜻을 가진 최소 단위	3
BLEU		n-gram 정밀도 기반 생성 품질 점수	9
Distinct-n		응답들이 서로 얼마나 다른지 재는 다양성 지표	9
EOS / BOS / PAD	Special Tokens	문장 끝/시작/빈칸 표시용 특수 토큰	3
n-gram		연속된 n개 토큰 묶음	9
OOV	Out-Of-Vocabulary	모델이 모르는 단어	3
ROUGE-L		최장 공통 순서 기반 생성 품질 점수	9
Transformer		어텐션 기반의 현대 언어 모델 표준 구조	5
vocab	Vocabulary	모델이 아는 토큰 전체 목록	3
Word2Vec		주변 단어 관계로 임베딩을 학습하는 방법	4

이 문서의 모든 수치는 2026-09-19 실행 기준이며, 노트북 01~08의 출력 및 `report/결과보고서.pdf`와 일치합니다. 더 자세한 실험 설계와 근거는 결과보고서를, 실제 코드는 각 노트북을 참고하세요.